

Oct 7, 2026

Auction Management, Media Uploads & Server-Side Logging

Overview

In this task, you will create an auction system where sellers can create and manage auctions, while buyers can browse available auctions. You will also implement **role-based access control (RBAC)** to make sure users can only perform actions allowed for their role.

You will learn how to handle image uploads using **Multer** and store uploaded images using **Cloudinary**. You will also learn **server-side logging** using **Morgan** for HTTP request logging and **Pino with pino-pretty** for application logs.

By the end of this task, your backend will have a working auction management system with secure role-based permissions, cloud-based image storage, and structured server-side logging.

Goals

Auction Management

- Create an Auction model with fields such as title, description, category, starting price, images, start/end time, status, and seller.
- Implement APIs to create, view, update, and delete auctions.
- Associate each auction with the seller who created it.

Role-Based Access Control & Ownership

- Implement **Buyer, Seller, and Admin** roles with different permissions.
- Allow sellers to manage only their own auctions, while admins can manage all auctions.

- Prevent unauthorized users from performing restricted actions.

Auction Status & Lifecycle

- Add basic status handling for auctions: UPCOMING, ACTIVE, ENDED
- The status should depend on the auction's startTime and endTime.

Image Uploads & Cloudinary

- Use Multer to handle auction image uploads.
- Upload images to Cloudinary and store their URLs in MongoDB.
- Support multiple images for an auction.

API Testing & Validation

- Test all auction APIs and authorization scenarios using Postman.
- Add validation for auction fields, prices, and timings.
- Include Postman screenshots in the README.

MVC Logging & Code Organization

- Continue following the MVC architecture from Task 2 and keep routes, controllers, models, and middleware properly separated.
- Use **Morgan** for HTTP request logging and **Pino with pino-pretty** for application/server logs.
- Reuse the authentication middleware from Task 2.

Useful Tips

- Always check the user's **role and ownership** before allowing auction operations.
- Take the **seller ID from the authenticated user**, not from the request body.
- Do not store image files directly in MongoDB, store the **Cloudinary URLs** instead.

Learning Resources

Reference Links

1. **Multer :**
    a. https://www.youtube.com/watch?v=i8yxx6V9UdM
    b. https://www.youtube.com/watch?v=6KPXn2Ha0cM
2. **Cloudinary** - https://www.youtube.com/watch?v=3Gj_mL9JJ6k
3. **Morgan -** https://www.youtube.com/watch?v=F2eQRlBX6Hs

Documentation

1. **Multer** - https://medium.com/@mohsinansari.dev/handling-file-uploads-and-file-validations-in-node-js-with-multer-a3716ec528a3
2. **Cloudinary -** https://cloudinary.com/blog/uploading-images-node-js-cloudinary-node-sdk
3. **CRUD -** https://medium.com/@skhans/how-to-build-a-basic-node-js-crud-app-with-mongoose-and-mongodb-3e958a36001d
4. **Morgan -** https://www.npmjs.com/package/morgan
5. **Pino-Pretty -** https://www.npmjs.com/package/pino-pretty